

中山大学网络空间安全学院必修课程

第4节 选择结构

C Programming

岳 晟

网络空间安全学院 工学园2栋508

yuesh5@mail.sysu.edu.cn

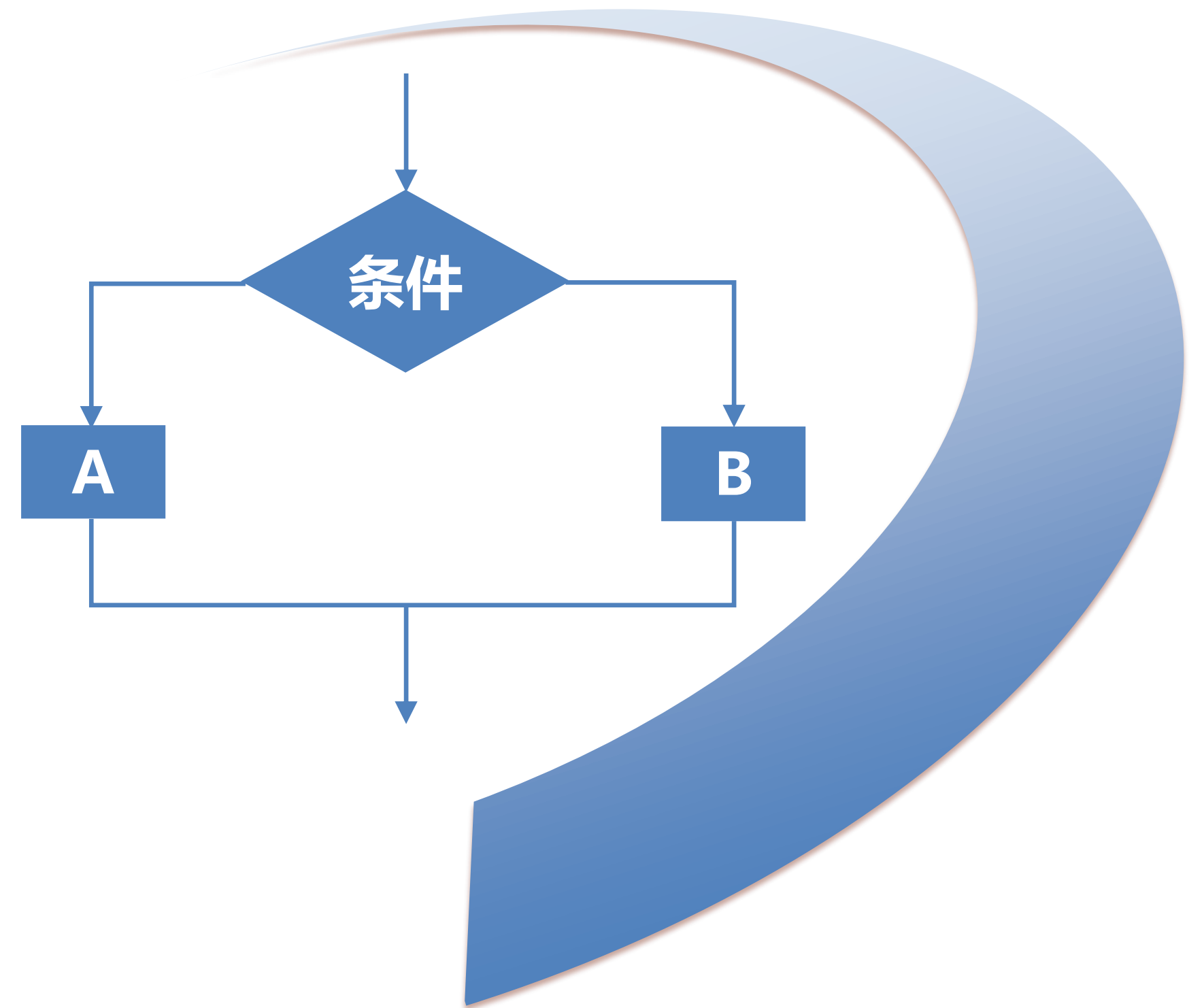
2026年秋季

结构化程序设计 (Structured Programming Theorem)

- **任何**复杂的程序逻辑，都可以用**三种基本结构**（程序流）组合而成：
 - 1. 顺序结构**：程序按照代码书写的先后顺序，自上而下逐条执行
 - 2. 选择结构**：根据条件判断的结果，选择执行不同的代码路径
 - 3. 循环结构**：在满足条件的情况下，重复执行一段代码

本节内容

1. if 语句
2. 条件表达式
3. switch 语句



求 $ax^2 + bx + c = 0$ 的根

```
#include <stdio.h>
#include <math.h>

int main(void){
    float a, b, c;
    float disc, x1, x2, p, q;

    scanf("%f%f%f", &a, &b, &c);

    disc = b * b - 4 * a * c;
    x1 = (-b + sqrtf(disc)) / (2.0 * a);
    x2 = (-b - sqrtf(disc)) / (2.0 * a);

    printf("x1=%.2f\nx2=%.2f\n", x1, x2);

    return 0;
}
```

求 $ax^2 + bx + c = 0$ 的根

- 假设 a 、 b 、 c 的值并不保证判别式 $b^2 - 4ac \geq 0$ ，需要在程序中**判断**：
 - 如果 $b^2 - 4ac \geq 0$ ，就输出方程的两个实根
 - 如果 $b^2 - 4ac < 0$ ，就输出“无实根”的信息

求 $ax^2 + bx + c = 0$ 的根

if 语句

```
#include <stdio.h>
#include <math.h>
int main(void){
    float a, b, c, disc, x1, x2, p, q;
    scanf("%f%f%f", &a, &b, &c);
    disc = b * b - 4 * a * c;

    if (disc < 0) {
        printf("No real roots!\n");
    }
    else {
        x1 = (-b + sqrtf(disc)) / (2.0 * a);
        x2 = (-b - sqrtf(disc)) / (2.0 * a);
        printf("x1=%7.2f\nx2=%7.2f\n", x1, x2);
    }
    return 0;
}
```

if 语句

```
if (表达式)  
    语句1  
[else  
    语句2]
```

- 若表达式为**真 (非0)**，执行语句1；否则，跳过语句1
- 语句1和语句2可以是一个**普通语句**，也可以是由 **{ }** 包围的一个**复合语句**
- [] 表示**else子句可选**

关系运算符与关系表达式

□ 关系运算符用于**比较**两个值之间的关系，表达式结果为整数 **0** 或 **1**：
成立为 1（真），不成立为 0（假）

运算符	含义	a=10, b=20
==	判断两边的值是否 相等	a == b → 0 (假)
!=	判断两边的值是否 不相等	a != b → 1 (真)
>	判断左边的值是否 大于 右边	a > b → 0 (假)
<	判断左边的值是否 小于 右边	a < b → 1 (真)
>=	判断左边的值是否 大于等于 右边	a >= b → 0 (假)
<=	判断左边的值是否 小于等于 右边	a <= b → 1 (真)

关系运算符优先级



$c > a + b$ 等价于 $c > (a + b)$

$a > b == c$ 等价于 $(a > b) == c$

$a == b < c$ 等价于 $a == (b < c)$

$a = b > c$ 等价于 $a = (b > c)$

练习

□ 若 $a=3$ 、 $b=2$ 、 $c=1$ ，则以下表达式的值是？

1. $d = a > b$

2. $f = a > b > c$

逻辑运算符与逻辑表达式

□ 逻辑运算符用于对操作数的真假状态进行**逻辑运算**，运算结果为整数 0 或 1：结果**成立时为 1（真）**，结果**不成立时为 0（假）**

运算符	描述	a=1, b=0
&&	逻辑与 (AND) ：两个操作数 都为非 0 时，结果为 1；否则结果为 0	$a \ \&\& \ b \rightarrow 0$
	逻辑或 (OR) ：两个操作数中 至少有一个为非 0 时，结果为 1；两个操作数都为 0 时，结果为 0	$a \ \ b \rightarrow 1$
!	逻辑非 (NOT) ：对操作数的 真假状态取反 ；操作数为 0 时结果为 1，操作数为非 0 时结果为 0	$!(a \ \&\& \ b) \rightarrow 1$

逻辑运算符与逻辑表达式

a	b	!a	!b	a && b	a b
真 (非0)	真 (非0)	假 (0)	假 (0)	真 (1)	真 (1)
真 (非0)	假 (0)	假 (0)	真 (1)	假 (0)	真 (1)
假 (0)	真 (非0)	真 (1)	假 (0)	假 (0)	真 (1)
假 (0)	假 (0)	真 (1)	真 (1)	假 (0)	假 (0)

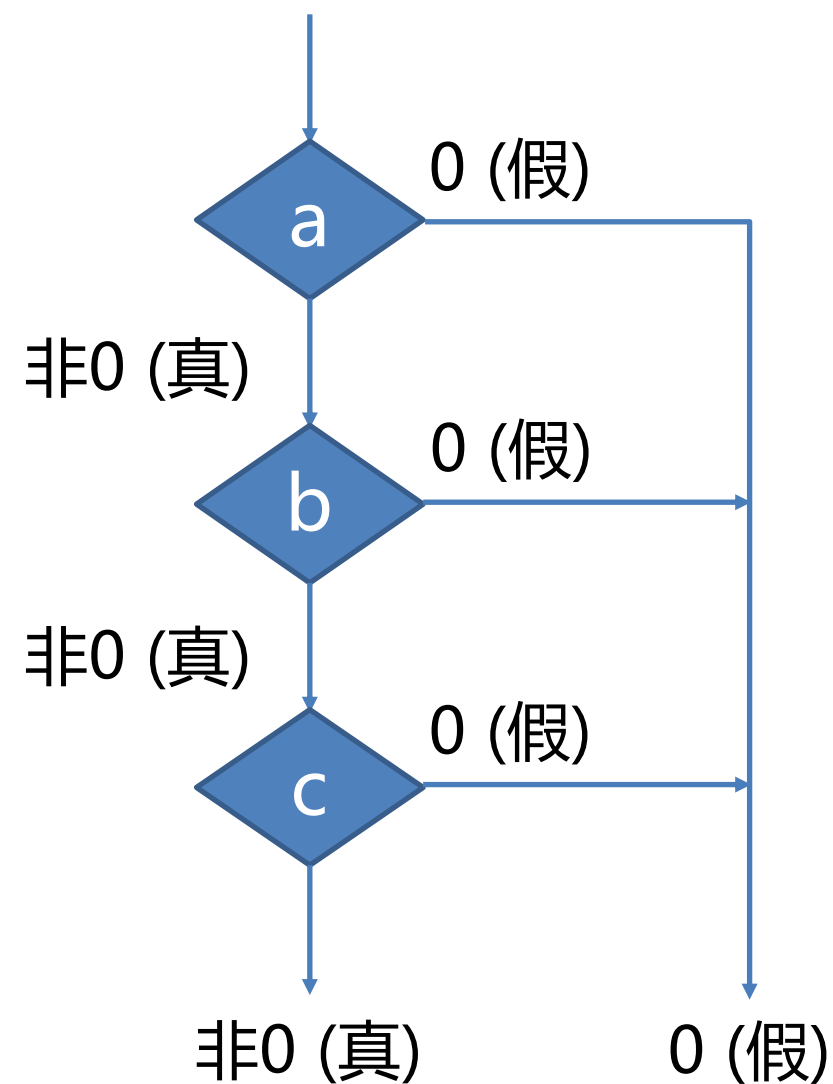
优先级：! → && → ||

“&&” 和 “||” 低于关系运算符，“!” 高于算术运算符

逻辑运算短路

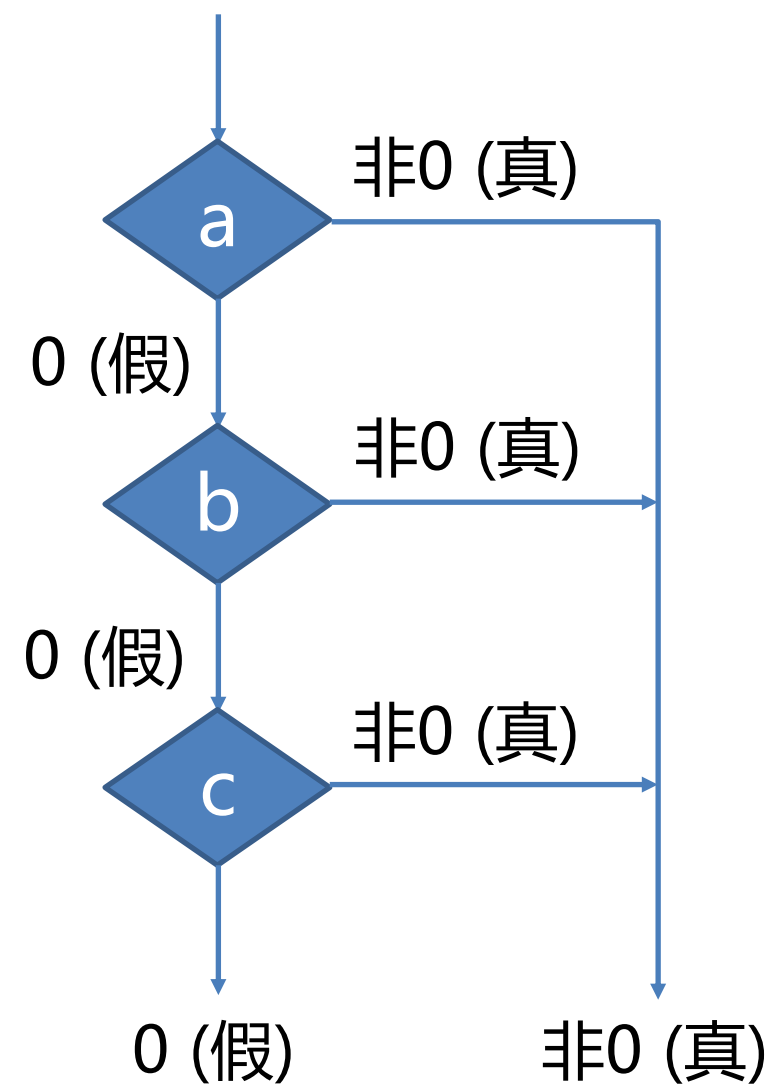
□ 在逻辑表达式求解中，只计算确定结果所必需的部分

- **a && b && c** 只有a为真 (非0) 时，才需要判别b的值；只有当a和b都为真时才需要判别c的值



逻辑
表达式

- **a || b || c** 只要a为真 (非0)，就不必判断b和c；只有a为假，才判别b；a和b都为假才判别c



思考：以下程序会输出什么？

□ 为提升程序性能：

- 在 and 表达式中把大概率为 false 的子表达式写在左边
- 在 or 表达式把大概率为 true 的子表达式写在左边

```
int a, b, c, d;  
a = b = c = -1;  
d = ++a && ++b && ++c;  
printf("%d %d %d %d\n", a, b, c, d);
```

```
int a, b, c, d;  
a = b = c = 0;  
d = a++ || ++b || c++;  
printf("%d %d %d %d\n", a, b, c, d);
```

if 语句常见形式

if (表达式)
语句1
[else
语句2]

形式1：没有else子句部分

if (表达式) 语句1

形式2：有else子句部分

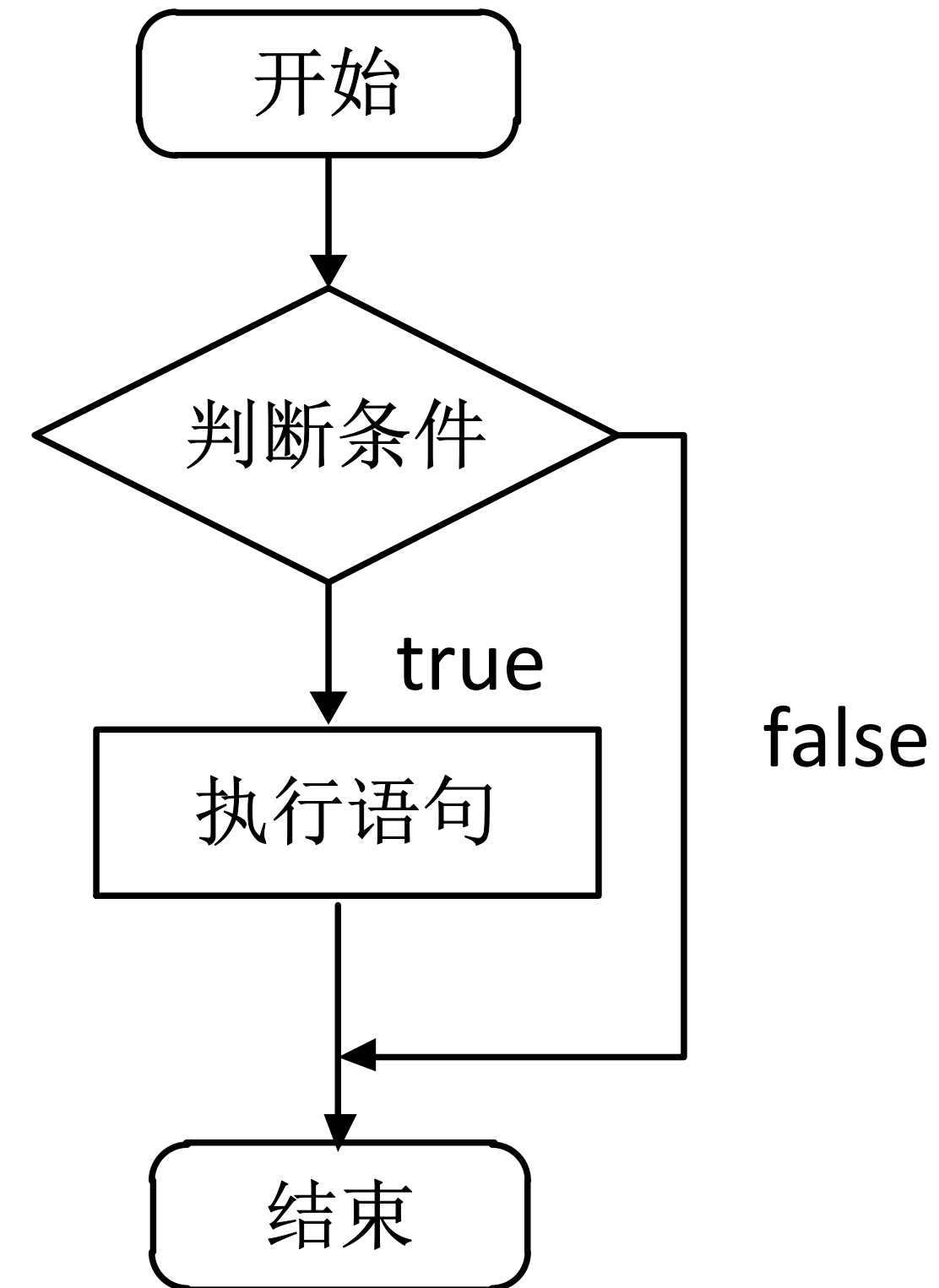
if (表达式) 语句1
else 语句2

形式3：在else部分又嵌套了多层的if语句

```
if (表达式1)          语句1
else if (表达式2)     语句2
else if (表达式3)     语句3
      ⋮                ⋮
else if (表达式m)     语句m
else                  语句m+1
```

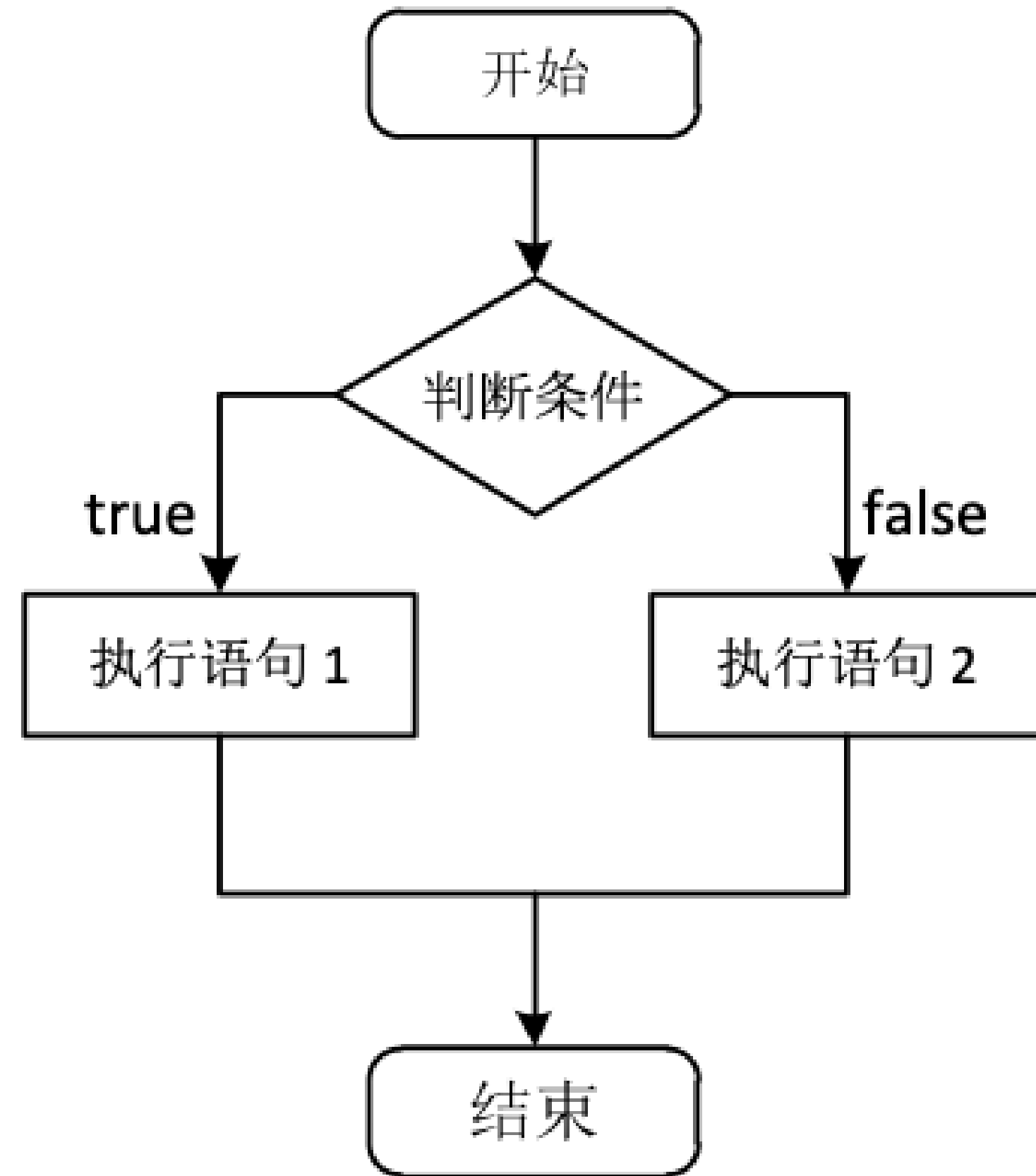
形式1：没有else子句部分

if (表达式)
语句1



形式2：有else子句部分

```
if (表达式)  
    语句1  
else  
    语句2
```



形式2：有else子句部分

- 输入两个数a和b，输出a和b之间的最大值

```
#include <stdio.h>

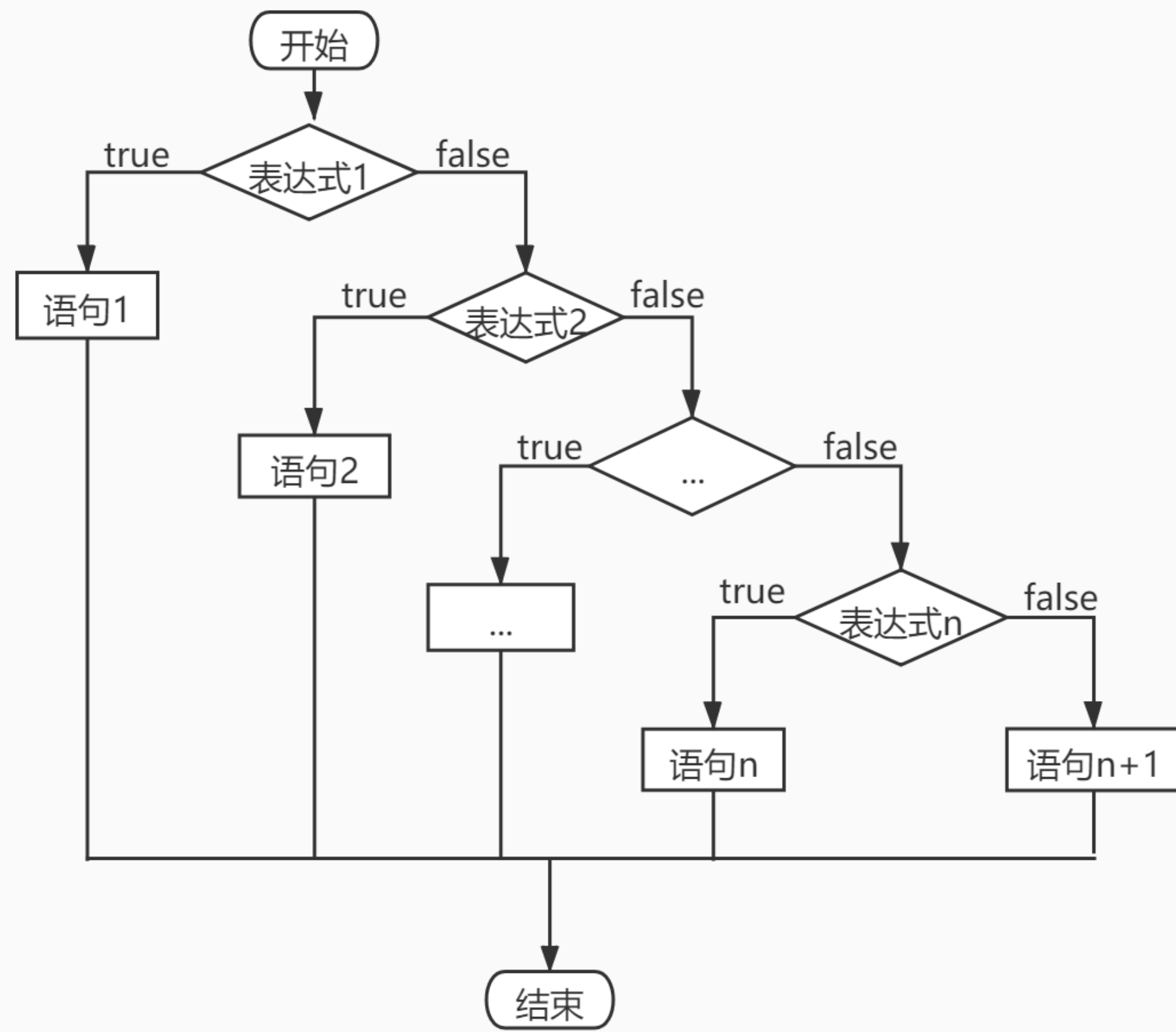
int main(void) {
    int a, b, max;
    printf("输入两个整数: ");
    scanf("%d%d", &a, &b);

    if (a > b) max = a;
    else max = b;

    printf("%d和%d的较大值是: %d\n", a, b, max);
    return 0;
}
```

形式3：有else子句部分

```
if (表达式1)
    语句1
else if (表达式2)
    语句2
    ⋮
else if (表达式m)
    语句m
else
    语句m+1
```



形式3：有else子句部分

```
if (表达式1)
    语句1
else if (表达式2)
    语句2
    :
else if (表达式m)
    语句m
else
    语句m+1
```

1. 首先判断表达式1的值是否为真，如果为真，那么执行下面的语句1，之后跳过剩下的判断
2. 否则，判断表达式2的值是否为真，如果为真，那么执行下面的语句2，之后跳过剩下的判断
3. 继续判断下去.....直到找到一个表达式的值为真，执行后面的语句
4. 否则在最后的else分支中，执行语句m+1

例：判断成绩等级

□ 输入一个 0 ~ 100 的整数成绩 score，输出对应等级：

A: 90 ~ 100

B: 80 ~ 89

C: 70 ~ 79

D: 60 ~ 69

F: 0 ~ 59

```
#include <stdio.h>

int main(void) {
    int score;
    scanf("%d", &score);

    if (score >= 90) printf("A\n");
    else if (score >= 80) printf("B\n");
    else if (score >= 70) printf("C\n");
    else if (score >= 60) printf("D\n");
    else printf("F\n");

    return 0;
}
```

if 语句常见形式

if (表达式)
语句1
[else
语句2]

形式1：没有else子句部分

if (表达式) 语句1

形式2：有else子句部分

if (表达式) 语句1
else 语句2

形式3：在else部分又嵌套了多层的if语句

if (表达式1) 语句1
else if (表达式2) 语句2
else if (表达式3) 语句3
⋮
else if (表达式m) 语句m
else 语句m+1

思考

□ 以下程序何时输出 “You're so good!” ？

```
#include <stdio.h>

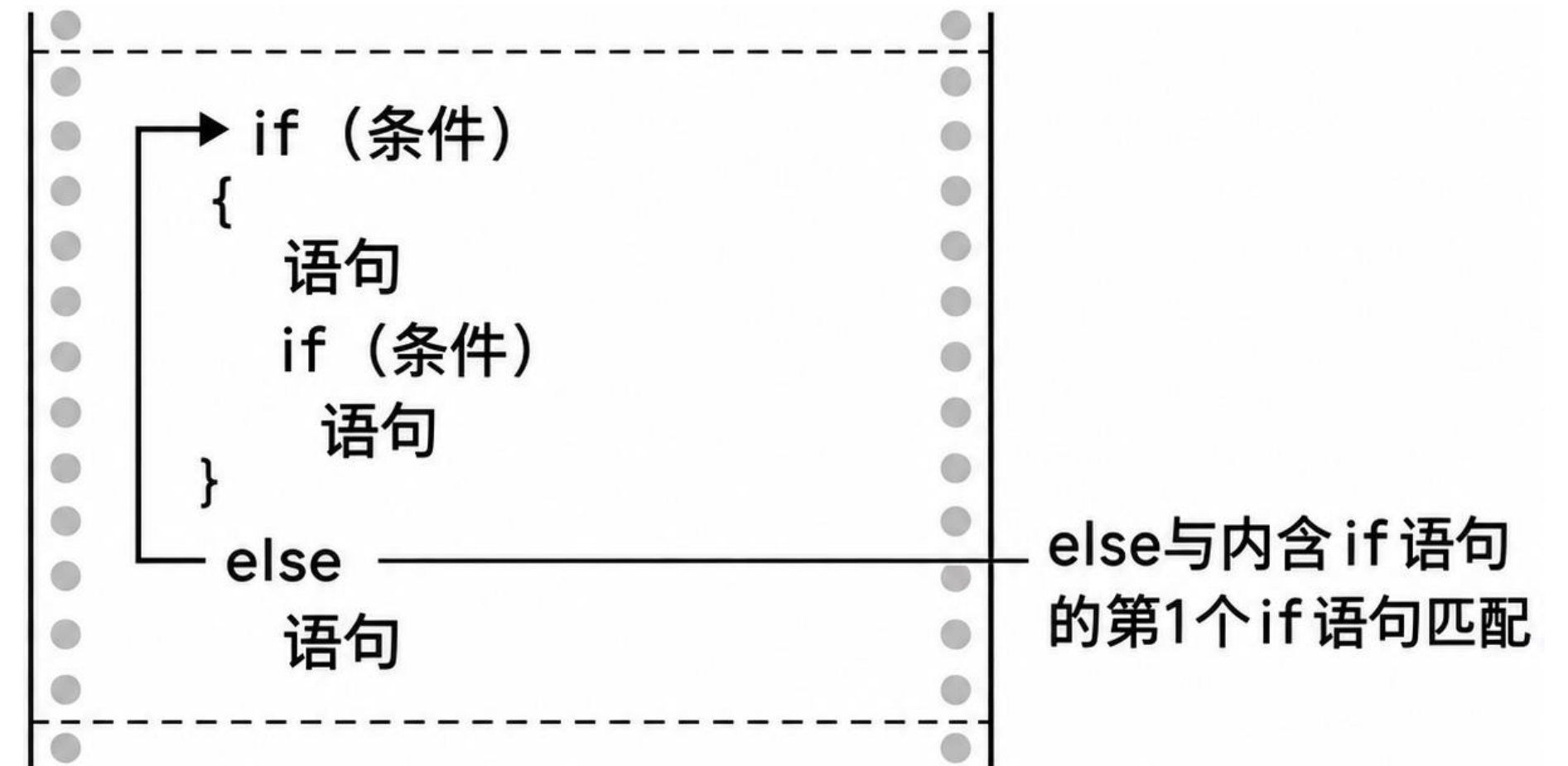
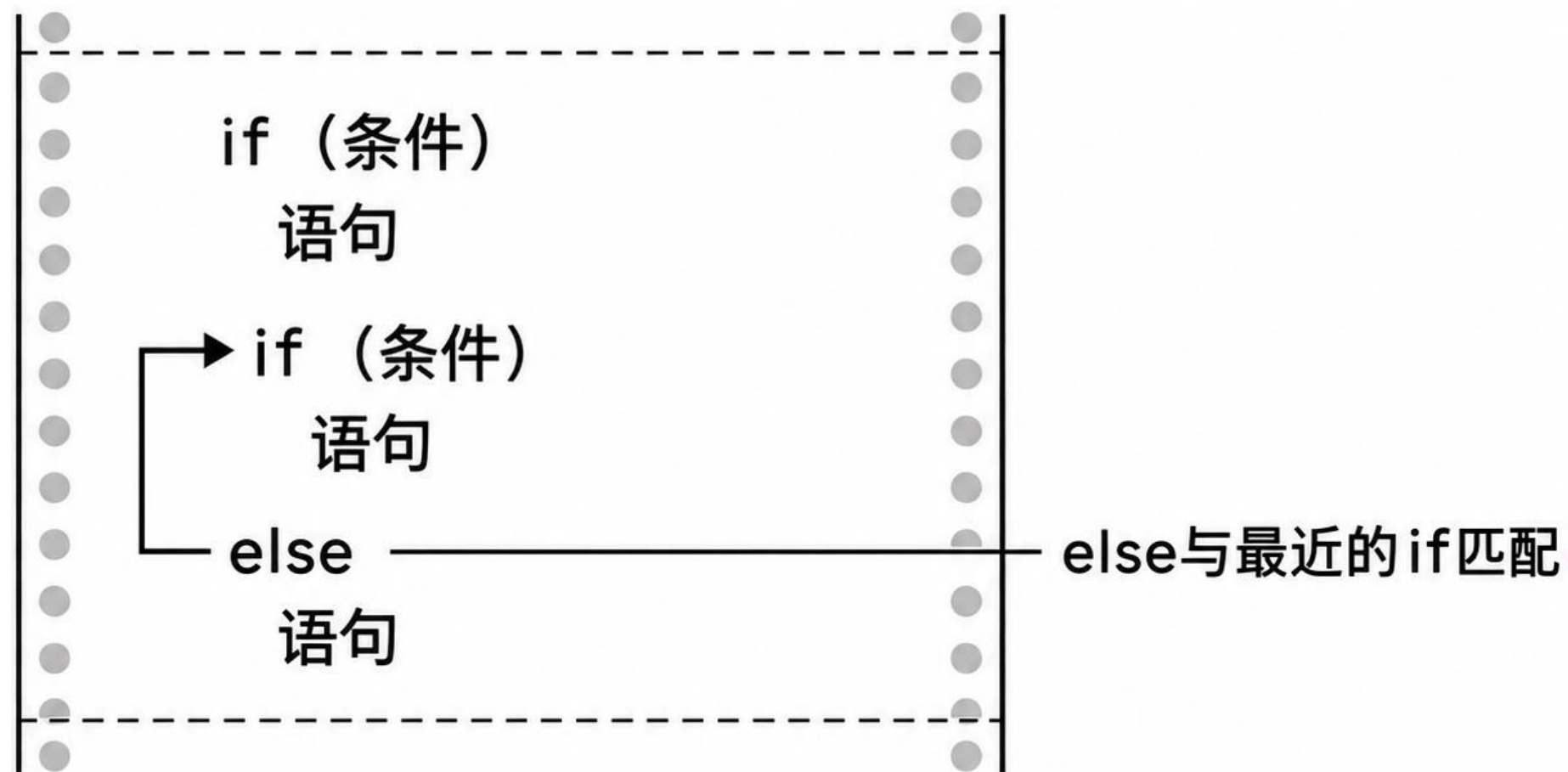
int main(void) {
    int score;
    scanf("%d", &score);

    if (score < 90)
        if (score > 60)
            printf("Need to work harder!\n");
        else
            printf("You're so good!\n");

    return 0;
}
```

else 与 if 的配对关系

- 如果没有花括号，else 与离它最近的未配对的 if 匹配



练习

- 判断年份是否为闰年，判断规则：能被 4 整除，但不能被 100 整除，如 2008；或者能被 400 整除，如 2000

```
int leap;
if (year % 4 == 0) {
    if (year % 100 == 0) {
        if (year % 400 == 0)
            leap = 1;
        else
            leap = 0;
    }
    else {
        leap = 1;
    }
}
else {
    leap = 0;
}
```

解法1

```
int leap;

if (year % 4 != 0)
    leap = 0;
else if (year % 100 != 0)
    leap = 1;
else if (year % 400 != 0)
    leap = 0;
else
    leap = 1;
```

解法2

```
int leap;
leap = (year%4==0 && year%100!=0) || year%400==0;
```

解法3

条件运算符和条件表达式

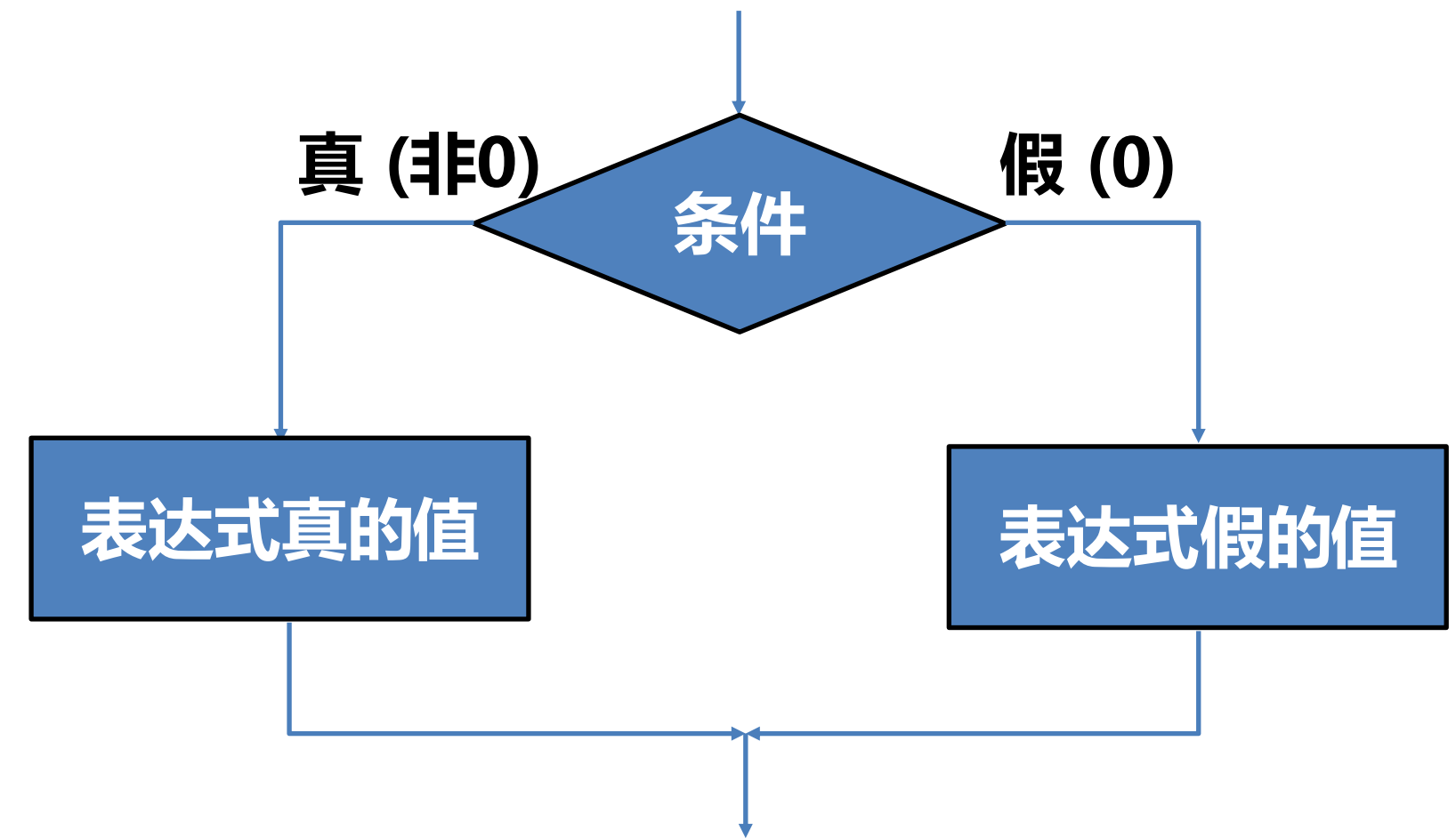
```
if (a > b)
    max = a;
else
    max = b;
```



```
max=(a>b) ? a : b;
```

- ❑ 条件运算符由 **?** 和 **:** 组成，须一起使用，要求有3个操作对象，故也称**三元运算符**
- ❑ 计算条件的值是否为真：
 - 如果为**真 (非0)**，那么求解**表达式真**，其值作为整个条件表达式的值
 - 如果为**假 (0)**，那么求解**表达式假**，其值作为整个条件表达式的值

条件 ? 表达式真 : 表达式假



注意：两个表达式须类型兼容，若类型不同，会发生隐式转换

练习

□ 输入两个数a、b，输出a、b之间的最大值

```
max=(a>b) ? a : b;
```

□ 输入三个数a、b、c，输出a、b、c之间的最大值

```
max = (a > b) ? ((a > c) ? a : c) : ((b > c) ? b : c);
```

问题

- 输入 1-7 之间的正整数，输出正整数对应是星期几

```
#include <stdio.h>
int main(void) {
    int a;
    scanf("%d", &a);

    if (a == 1) {
        printf("Monday\n");
    } else if (a == 2) {
        printf("Tuesday\n");
    } else if (a == 3) {
        printf("Wednesday\n");
    } else if (a == 4) {
        printf("Thursday\n");
    } else if (a == 5) {
        printf("Friday\n");
    } else if (a == 6) {
        printf("Saturday\n");
    } else if (a == 7) {
        printf("Sunday\n");
    } else {
        printf("error\n");
    }

    return 0;
}
```

多分支选择：switch 语句

```
switch(表达式)
{
    case 常量1: 语句1; [break];
    case 常量2: 语句2; [break];
    case 常量3: 语句3; [break];
    ...
    case 常量n: 语句n; [break];
    default: 语句n+1
}
```

- ❑ 执行 switch 结构：先计算 switch 后面“表达式”的值，然后将它逐个与 case 语句中的常量进行比对
- ❑ 如果某一个 case 语句中的常量相同，流程就转到此分支，执行此 case 后面的语句
- ❑ break 是 C 语言中的一个关键字，可以跳出 switch 语句

多分支选择：switch 语句

```
switch(表达式)
{
    case 常量1: 语句1; [break];
    case 常量2: 语句2; [break];
    case 常量3: 语句3; [break];
    ...
    case 常量n: 语句n; [break];
    default: 语句n+1
}
```

- 所谓“跳出”，是指一旦遇到 break，就**不再执行 switch 中的任何语句**，跳转到 switch 语句末尾右花括号处，接着执行后面的代码
- 如果没有与 switch 表达式相匹配的 case 标签，流程会进入到 **default** 分支，执行 default 标签后面的语句；如此时没有 default 标签，则 switch 结构不执行任何语句

多分支选择: switch 语句

```
switch(表达式)
{
    case 常量1: 语句1; [break];
    case 常量2: 语句2; [break];
    case 常量3: 语句3; [break];
    ...
    case 常量n: 语句n; [break];
    default: 语句n+1
}
```

```
#include <stdio.h>
int main(void) {
    int a;
    printf("Input integer number:");
    scanf("%d",&a);
    switch(a){
        case 1: printf("Monday\n");      break;
        case 2: printf("Tuesday\n");     break;
        case 3: printf("Wednesday\n");  break;
        case 4: printf("Thursday\n");    break;
        case 5: printf("Friday\n");      break;
        case 6: printf("Saturday\n");    break;
        case 7: printf("Sunday\n");      break;
        default: printf("error\n");      break;
    }
    return 0;
}
```

switch语句

□ 注意:

1. 测试表达式的值应该是一个**整型值**
2. case 标签必须是**整型常量表达式**

```
case 10: printf("..."); break;           //正确
case 8+9: printf("..."); break;          //正确
case 'A': printf("..."); break;          //正确, 字符和整数可以相互转换
case 'A'+19: printf("..."); break;       //正确, 字符和整数可以相互转换
case 9.5: printf("..."); break;          //错误, 不能为小数
case a: printf("..."); break;            //错误, 不能包含变量
case a+10: printf("..."); break;         //错误, 不能包含变量
```

总结

选择语句

```
if (表达式) 语句1  
[else 语句2]
```

条件表达式

```
条件 ? 表达式真 : 表达式假
```

多分支选择语句

```
switch(表达式)  
{  
    case 常量1: 语句1; [break];  
    case 常量2: 语句2; [break];  
    case 常量3: 语句3; [break];  
    ...  
    case 常量n: 语句n; [break];  
    default: 语句n+1  
}
```

作业

- 阅读《C程序设计》第4章
- 完成 PTA 本周习题

谢谢